

Speedway Party

Contents

Game manual for the Amiga 1200

version a9acac0 2026-10-16 · generated 2026-10-16

Part I. For players

1. Requirements and installation
2. Controls
3. Game screens
4. Tournament rules
5. The tape start
6. Riding and the racing line

Part II. For developers

7. Building
8. Game architecture
9. Running in WinUAE
10. PC tools and tests
11. Technology stack
12. Status and known limitations
13. How to update this documentation
14. Credits and licence

An arcade speedway racing game for the Amiga 1200 (AGA). A 15-heat tournament for 1–4 players on one screen; empty places are taken by bots.

> This file is the source of the documentation. Edit it as plain text (Markdown) and make the PDF with `python tools/mkdocs.py` (see the chapter *How to update this documentation*). The version and date in the PDF come from git automatically. The Polish original is `docs/DOKUMENTACJA.md`.



A heat

Part I. For players

1. Requirements and installation

Requirements: an Amiga 1200 or another AGA machine, a 68020 CPU, 2 MB Chip RAM, Kickstart 3.x (the floppy boots from Kickstart 2.0 on). Fast RAM is not needed. PAL (50 Hz) display only.

The game comes in two languages – Polish and English – in three files (the `release/` directory in the repository, and on the GitHub Releases page):

File	What it is
`SpeedwayParty_EN.adf`	an 880 KB boot floppy (FastFileSystem) with the English version. Insert it in DF0: (or a Gotek, or an emulator) and the game starts by itself.
`SpeedwayParty_PL.adf`	the same in Polish.
`SpeedwayParty.zip`	both versions for a hard disk: the `SpeedwayParty/` drawer with its icon, the programs `SpeedwayParty_EN` and `SpeedwayParty_PL` with icons, shared data in `data/`, `s/startup-sequence` and a short manual `README.TXT` / `CZYTAJ.TXT` with icons.

Running the game:

- **From floppy:** insert `SpeedwayParty_EN.adf` in DF0: and switch the Amiga on (in WinUAE: *Floppy drives → DF0:*).
- **From a hard disk:** unpack the zip to the Amiga's disk and on Workbench double-click the `SpeedwayParty_EN` icon in the `SpeedwayParty` drawer – or in the Shell go to the `SpeedwayParty` directory and type `SpeedwayParty_EN`. The `data/` directory must sit next to the program. Both versions share the same records.
- **In an emulator, from a directory:** mount the unpacked directory as a boot disk (chapter 9). Its `s/startup-sequence` starts the Polish version; change it to `SpeedwayParty_EN` for English.

The game should run on:

- an Amiga 1200 – also a bare one without Fast RAM (2 MB Chip RAM) – from floppy or from a hard disk / CF card,
- an Amiga 1200 with an accelerator card (68030/040/060) and Fast RAM,
- an Amiga 4000 and other machines with the AGA chipset and Kickstart 3.0, 3.1 or 3.2,
- Amiga emulators set up as an A1200: WinUAE, FS-UAE, EmulatorJS (PUAE).

ESC (or the second joystick button) opens a pause with a question at any time: during a tournament – whether to go back to the menu, on the title screen – whether to quit the game. In the settings ESC goes straight back to the title screen. A second ESC means “yes”, FIRE returns to the game.

2. Controls

Each player has just two controls: **LEFT** and **RIGHT**. The throttle is automatic.

	Player 1	Player 2	Player 3	Player 4
default	joystick in port 1	joystick in port 0	keys Z / X	keys N / M
optional	cursor keys, 4-player adapter (parallel port)			

- The **JOY 1** and **CURSORS** settings work interchangeably (joystick in port 1 or cursor keys), unless another player has chosen the other one.
- **Menus:** joystick in port 1 or 0 (up/down/left/right, FIRE = confirm) or cursor keys and Return.
- **The second joystick button** works like ESC: pause with a question, back from the manual and the credits.
- **The joystick in port 0** must be set to joystick mode in the emulator, not mouse.
- **Names** are edited with the keyboard; Polish letters via Alt (e.g. Alt+X = Ź, Alt+Z = Ż).

3. Game screens

Title screen and main menu

Loading from floppy takes several dozen seconds, so the game first shows the title screen with “LOADING…” and reads the rest of the data in the background (the system still runs normally; the game takes over the hardware only once everything is loaded). The title screen waits for FIRE. Then you choose:

- **LET'S RACE!** – the tournament settings,
- **HOW TO PLAY** – three pages: tournament rules and flags, start and controls, the racing line with a track diagram (LEFT/RIGHT turns the page),
- **RECORDS** – the track record tables,
- **CREDITS** – who made the game.

FIRE or ESC takes you back from the manual, the records and the credits.



Main menu after the title screen

Settings

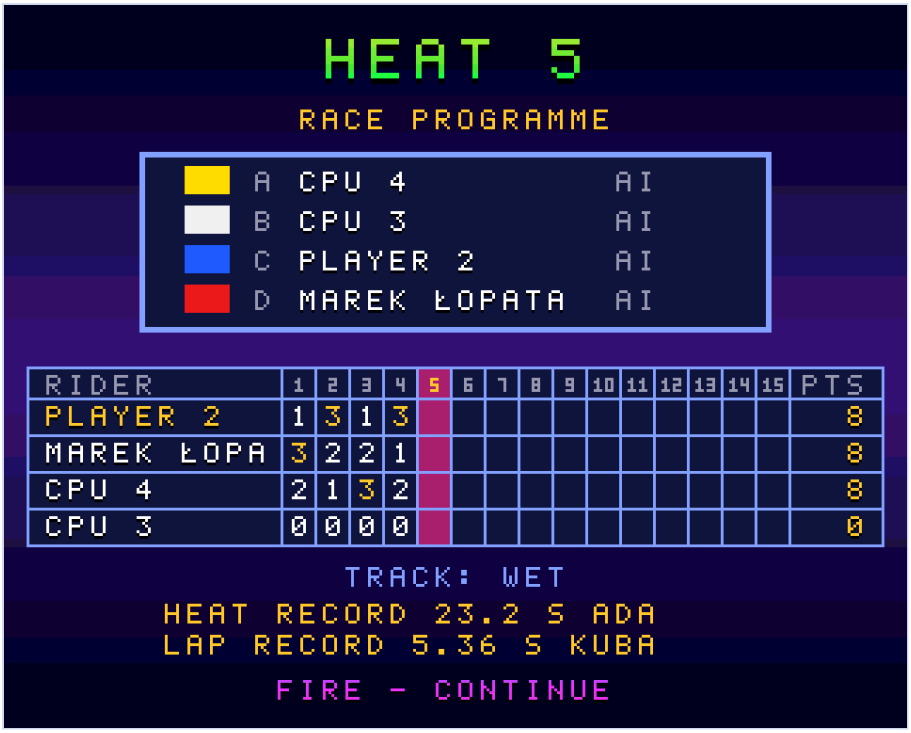
Number of players (0–4), each player's controls and name, bot skill, track condition (dry, damp, wet – random at first), music and defects (chapter 4). The cursor starts on **START TOURNAMENT**. PLAYER 1 gets the control you confirmed LET'S RACE! with: joystick in port 0 – JOY 0, in port 1 – JOY 1, keyboard (Return) – CURSORS (the player who had that control takes player 1's previous one). With 0 players the whole tournament is ridden by bots – a demo mode.



Settings

Race programme and rider presentation

Before each heat the programme shows who starts from which gate, the points table for all heats and the track records. Then the start girls stand at gates A–D holding boards in the helmet colours of the riders starting there (3 seconds or FIRE). They sway to the beat, and every now and then one of them blows a kiss.



Race programme



Rider presentation

The heat

At the bottom of the screen there is a bar: helmet colours in race order, the lap number, round start lights (they disappear 2 s after the start – not needed during the heat) and the referee with a flag. During the start, in the middle of the screen on the infield (above the riders on the straight), a starting gate window with a lamp and the tape is shown.



The tape start

Results, track break, podium and newspaper

After each heat – results with points and the time. After heats 4, 7, 11 and 13 tractors grade and water the track. After the last heat – the podium, followed by the “OVAL HERALD” newspaper summing up the meeting. FIRE on the newspaper starts a new tournament.



Heat result



A defect: the engine coughs and smokes (yellow bike)



The newspaper after the meeting

Track records

The track has two tables: **the 5 best heats** (time of all 4 laps) and **the 5 best laps**. After every heat, every player who finished with a good enough time gets in. Bots never enter the tables, so a bots-only demo (0 players) saves nothing. On an equal time the earlier entry stays higher.

A player who made the top five during the tournament enters – once, for all their places, after the meeting (after the newspaper); the game does not stop between heats – a **name of up to 7 characters** (by default the first word of the rider's name): UP/DOWN changes the character (space, A–Z, Polish letters, 0–9, `-`), LEFT/RIGHT the position; you can also type on the keyboard (Alt + letter = Polish letter, Backspace deletes). FIRE or Return saves, ESC keeps what is on the screen. When several players made the top five, they take turns. If the tournament is abandoned (ESC), its players' places get names taken from their rider names.

The tables are saved to `halloffame.dat` next to the game (the previous version is kept as `halloffame.bak`) when you quit the game: ESC on the title screen and ESC once more. Resetting or switching off the Amiga during play loses the session's results, and on a write-protected floppy saving silently fails. Player records from the old `records.dat` file are imported once, on the first start. The Polish and English versions share the same files.



Name entry



Track records

4. Tournament rules

- **15 heats of 4 laps, 4 riders each.** Points for places: 3, 2, 1, 0.
- **Everyone starts from every gate** – heats 1–13 are arranged that way.
- **Nominated heats (14 and 15):** the leader of the table picks a gate first, then the others.

- **Run-off:** a tie on points for a podium place is decided by an extra heat between the tied riders, with random gates. A tie outside the podium is decided by heat wins.
- **Defects** (an option in the settings, on by default): before every points heat each rider has a 2% chance of a defect – a machine failure – at most one per tournament; on average about one defect per tournament. At a random moment the engine starts to cough and smoke, dies after 1.5 s, and the bike rolls to a stop and leaves the track. The rider scores 0 points, shown as "D" in the table and "DEF" in the heat result. A rider who reaches the line before the engine dies finishes normally. There are no defects in a run-off.
- **Helmet colour** stays with a rider for the whole tournament (red, blue, white, yellow). The starting gate changes every heat.
- **Bots:** Rookie (late at the tape, slides too hard, hugs the kerb), Professional (quick reactions, "shuts the door" when you try to pass on the inside), Master (reads the track grip, doses the slide, can go wide).

Referee's flags

Flag	Meaning
chequered	end of the heat: from the leader entering the last straight until the last rider crosses the line
red	heat stopped, e.g. after someone touches the tape
black	rider excluded (e.g. a second tape offence in a heat)
yellow with a black cross	last lap
national	emergency flag start after a starting gate failure

When nothing is happening, the referee blinks and glances around from behind his tinted glasses.

5. The tape start

1. Hold LEFT or RIGHT while the riders are at the tape.
2. The red light is on – you wait one second, two, three, you never know how long.
3. Green, the buzzer, the snap of the released tape – **let go at once**. The faster you let go, the better the start.

- **Too early:** a whistle, "TAPE!", the red flag and a warning. A second tape offence in the same heat means exclusion.
- **Too late** (more than 0.3 s after green): the wheel spins in place.
- **Gate failure:** sometimes the lights go out and the referee starts the heat with the national flag – watch his hand.

6. Riding and the racing line

LEFT lays the bike into a slide (frame into the bend, at most 40°). **RIGHT** is the counter-steer – it straightens the bike (up to about 14°).

- **Every degree of slide costs speed.** A bike laid down fully turns sharply but loses a quarter of its speed. The fastest way round is to pulse: lay it down entering the bend, ease off in the middle, straighten on the way out.
- **The fence** takes a quarter of your speed, **the kerb** slows you slightly, **elbow-to-elbow contact** slows both riders.
- **Roost:** a sliding bike throws dirt from its rear wheel. Riding right behind a rival in his roost slows you down – look for a clean line.

The best line is different at every meeting

At the start of each tournament the track gets 1–2 worn-in, packed grooves at random places across its width. A groove shows as a darker band. On it:

- grip is better (faster acceleration, sharper turning),
- **sliding costs less speed** – up to 40% less in the middle of the groove; the further from it, the higher the penalty.

The track also wears in from heat to heat: the line the whole field rides gets darker and grips better, while the roost next to it loosens it. The grader wipes the grooves out during a track break.

Loose dirt and “going wide”

Loose dirt collects by the fence. Ride into it too slowly and the bike bogs down. Ride in at full speed in a light slide and the wide line “carries you” – the loose dirt speeds you up. A worn-in, packed groove by the fence no longer bogs you down. Exception: from the tape to the first bend everyone has the same grip (from the middle of the track) and nobody bogs down – gate D by the fence starts as fast as gate A.

Watering and drying

The track condition at the start is random (you can change it in the settings). A wet track grips clearly worse – you have to slide harder. After heats 4, 7, 11 and 13 the track is graded and watered, then dries out heat by heat.

Part II. For developers

7. Building

Toolchain

On Windows the easiest way is **WSL2 with Ubuntu**. The Bebbo toolchain is built once (about 30–60 min):

```
sudo apt update
sudo apt install make wget git gcc g++ lhasa libgmp-dev libmpfr-dev libmpc-dev flex bison gettext
texinfo ncurses-dev autoconf rsync libreadline-dev
git clone https://github.com/bebbo/amiga-gcc
cd amiga-gcc
make update
make all -j4 PREFIX=$HOME/amiga-gcc
echo 'export PATH=$HOME/amiga-gcc/bin:$PATH' >> ~/.bashrc
```

NDK 3.9 and libnix come with the toolchain. Check: `m68k-amigaos-gcc --version`.

Game, package and floppies

Command (in WSL, in the project directory)	Result
`make dist`	the `dist/` directory: program (Polish), `data/`, `s/startup-sequence` – to mount in an emulator. Bikes, tractors, the track background and the start girls are packed there (`tools/pack.py`, a simple LZ: 171 → 72 KB); the game unpacks them while loading. In the repository `data/` they are unpacked.
`make dist-en`	the `dist-en/` directory with the English version (`-DLANG_EN`, objects in `build/en/`)
`make release`	`release/SpeedwayParty.zip` (both versions) and the boot floppies `release/SpeedwayParty_PL.adf`, `release/SpeedwayParty_EN.adf`
`python tools/mkicons.py preview`	Workbench icons (game, drawer, disk, manual) – PNG preview
`make whdload`	a WHDLoad version in preparation (`dist-whd/`), not tested and not released yet
`make check`	syntax and type check of all the code with Clang for the 68020 (both languages)
`make sim`	a bot tournament simulator using the same game logic

Compiler flags: `-m68020 -mtune=68020 -O2 -fbbb=- -fomit-frame-pointer -noixemul`. The `-fbbb=-` option turns off Bebbo's extra optimizer, which lost a needed register reload in `switch` jump tables.

The ready data (`data/`) is in the repository – Python is not needed just to build the game.

Two languages

All texts shown by the game sit side by side in the code as `T("po polsku", "in English")` (macro in `src/game.h`). The language is chosen at compile time: by default Polish, with `-DLANG_EN` English. When you

add a new text, give both versions and check that the English one fits: `make check` compiles both, and `LANG_EN=1 sh tools/shots/run.sh` makes English screenshots in `art/en/`.

Menu music (build.cfg)

```
MUSIC = BlackMotor
```

Value	Track
`BlackMotor`	"Black Motor", 13.3 s, 208 KiB (default)
`NightGrid`	"Night Grid", 27.4 s, 428 KiB
`BRAK`	no music, saves Chip RAM

The tracks are in `src/music/<name>/` as `*_s8.raw` mixes (8-bit mono PCM, ~16 kHz). One-off: `make dist MUSIC=NightGrid`.

8. Game architecture

Main loop (50 Hz)

The game takes over the system (`system.c`) and runs on its own VBL interrupt. Every frame it:

1. waits for vertical blank (`system_wait_vbl`),
2. reads input and runs the logic (`update_state`: start, riding, bots, roost, tournament),
3. draws into the "back" buffer (`ui_render`, roost),
4. sets the sprites, updates the HUD and the sound,
5. waits for the Blitter and swaps buffers (`video_swap`).

All the work must fit in one frame: 313 PAL lines. The `make dist EXTRA=-DSHOW_LAG` build shows dropped frames and the time of each part in raster lines in the HUD.

Display

- **Playfield:** 320×208, 8 bitplanes (256 colours, 24-bit AGA palette), interleaved bitplanes, double buffered. Underneath there is the track background (`g_video.background`) used to restore areas.
- **HUD:** 320×48, 5 bitplanes (32 colours), single buffer. The referee, the hand with the flag (two waving phases kept in a buffer), helmet colours, round lights.
- **Copper:** per-line colours (menu background gradient, rainbow text, the newspaper's cream paper), separate lists for the race and the menus.
- **Sprites:** the bikes are attached 32 px pairs (15 colours); each bike has its own copy of the frame for each buffer, and the horizontal position is set every 1/4 pixel. Frames: 4 helmet colours × 32 directions × 3 slide angles.
- **Blitter:** fills, copies, roost dots ("cookie-cut"). Before drawing into a buffer the CPU waits for the blit to finish (`hw_wait_blit` / `blit_wait`).

Performance on an A1200 without Fast RAM

On a bare A1200 the program and data live in Chip RAM, shared with the display hardware. The rules that keep it at 50 frames per second:

- menu screens are redrawn only when something changes; a blinking text refreshes just its strip,
- worn-in grooves are repainted between heats (a few rows per frame); during a heat only the grip changes,
- blending the track palette (drying, watering) goes 64 colours per frame,
- the starting gate window and the flag phases are drawn once and copied from memory,
- costly drawing is done byte-wise instead of pixel by pixel in 8 planes.

Sound (Paula)

Four channels: two engines (the two highest-placed riders still racing; in the menu – double-buffered music), the crowd, effects (buzzer with the tape snap, whistle, the crowd's cheer when the leader enters the last straight – about 0.6 s before the line). The engine is an 8-firing-cycle loop from a recording (`data/engine.raw`); its pitch follows the RPM through Paula's playback period. The cheer (`data/cheer.raw`, 2.6 s) is generated by `tools/genaudio.py`; if a `crowd_cheer.wav` recording is in `src/`, the cheer is made from it (mono, 11,025 Hz, at most 3 s).

Modules

File	Responsible for
`main.c`	the 50 Hz loop, the tournament state machine
`system.c`	taking over and returning the hardware, the VBL interrupt
`video.c`	Copper lists, palettes, double buffering, HUD
`sprites.c`	frames (one set, helmet colour applied while copying) and bike sprite positions
`blit.c`	Blitter operations
`start.c`	start procedure, tape, reactions
`race.c`	riding model: throttle, slide, fence, contact, laps
`ai.c`	bots
`roost.c`	roost and water from the bowser
`track.c`	oval geometry, grip, moisture, worn-in grooves
`tractors.c`	track break
`tournament.c`	15-heat programme, points, nominated heats and run-offs, facts for the newspaper
`newspaper.c`	newspaper after the meeting: summary and headline choice
`ui.c`, `text.c`	screens, the font with Polish letters, all game texts in both languages
`hud.c`	the bottom bar, the referee
`audio.c`	Paula: engines, crowd, effects, music
`input.c`	joysticks, keyboard, parallel port adapter
`assets.c`, `records.c`, `hof.c`	loading data (and unpacking files made by `tools/pack.py`); TOP 5 track records (`halloffame.dat`: saving, backup, importing the old `records.dat`; table logic in `hof.c`)

Data (the data/ directory)

All files are made by the tools in `tools/` and are in the repository: track (`track0.bpl`, `track0.pal`), title screen (`title.bpl`, `title.pal`), bikes (`bikes.spr`: one set of frames, the game applies the helmet colour), tractors (`tractors.spr`), HUD (`hud.pal`, `referee.bin`, `refarm.bin`), start girls (`girls.bin`), sounds (`engine.raw`, `crowd.raw`, `buzzer.raw`, `whistle.raw`, `cheer.raw`) and music (`music.raw`, copied at build time).

9. Running in WinUAE

A ready configuration: `tools/winuae/SpeedwayParty-A1200.uae` (put the path to your `dist/` or `dist-en/` in it).

1. Model A1200, Kickstart 3.0/3.1 (your own ROM, e.g. from Amiga Forever) or AROS.
2. CPU 68020 *cycle-exact*, no FPU.
3. RAM: Chip 2 MB, Slow 0, Fast 0.
4. AGA chipset, PAL.

5. *CD & Hard drives → Add Directory*: the `dist/` (or `dist-en/`) directory, *Device name* `DH0`, *Boot priority* 0 or higher.
6. *Game ports:* port 1 = joystick (e.g. cursor keys), port 0 = a second joystick (e.g. a USB pad or joystick) in **Joystick** mode, not mouse. Don't leave a mouse in port 0.

Troubleshooting

Symptom	Cause / fix
Simple shapes instead of graphics	no `data/` directory next to the program
No sound	no `data/*.raw`
No music in the menu	`MUSIC = BRAK`, no `data/music.raw` or MUSIC: NO in the settings
The joystick in port 0 (JOY 0) only registers left, not right, and “moves” when released to centre	the emulator feeds port 0 as a mouse: set port 0 to **Joystick** mode (WinUAE: `joyport0mode=djoy`, as in the template; similarly in other emulators)
The menu changes by itself	a mouse in port 0
Cursor keys don't work, the heat waits at the start	in the WinUAE template the cursor keys emulate the joystick in port 1 – choose JOY 1 or CURSORS (both read both sources)
Garbled picture	`DDF_STOP` and `MOD_ADJ` in `src/video.c`

10. PC tools and tests

They need Python 3 with `numpy`, ffmpeg in PATH, and for the code check and the simulator `pip install ziglang`.

Command	What it does
`make assets`	regenerates graphics, sound and the font (`genart` , `gentitle` , `gengirls` , `genaudio` , `genfont`)
`python tools/gentitle.py`	the title screen from the illustration `src/title.png`
`python tools/genreferee.py`	the referee's portrait and painted hand from `src/referee.png`
`python tools/gengirls.py`	the start girls from `src/girls.png`
`python tools/import_engine.py file.wav`	the engine loop from a recording (currently `src/speedway_500cc_10000rpm_loop.wav`)
`sh tools/shots/run.sh`	screenshots from the real game code compiled for the PC (`art/screen_*.png` ; with `LANG_EN=1` English ones in `art/en/`)
`sh tools/mktrailer.sh`	the trailer `art/trailer.mp4`
`sh tools/sim/news_test.sh`	newspaper tests (headline rules, summary, long names)
`sh tools/sim/records_test.sh`	tests of the record tables and the `halloffame.dat` file
`sh tools/sim/sprites_test.sh`	bike frame test (helmet colour applied while copying = the old 4 sets)
`sh tools/sim/start_test.sh [tournaments]`	start fairness: 4 identical bots, results by gate A–D (speed after 1 s, place in the 1st bend and at the finish, wins); `DIAG=1` – roost, kerb, racing line
`sh tools/sim/pack_test.sh`	test of unpacking the data packed on the floppy
`sh tools/sim/batch.sh`	several bot tournaments – AI skill statistics
`python tools/mksocial.py`	the repository thumbnail on GitHub
`python tools/mkdocs.py`	this documentation as PDF (Polish and English)
`make dist EXTRA=-DAUTOPLAY`	a test build: goes through the menus by itself and plays a bot tournament
`make dist EXTRA=-DSHOW_LAG`	measurement: dropped frames and timings in the HUD
`make dist EXTRA="-DSHOW_LAG -DSHOW_START"`	measurement of the tape start alone

The track geometry lives in two places: `track_defs` in `src/track.c` and `PRESETS` in `tools/genart.py` – the values must be identical.

11. Technology stack

Area	What we use
Platform	Amiga 1200: 68020, AGA, 2 MB Chip RAM, no Fast RAM, PAL 50 Hz
Language	C (gnu99), no game libraries – direct programming of the Copper, Blitter, sprites, Paula and CIA
Compiler	Bebbo amiga-gcc 6.5.0b, NDK 3.9, libnix, make in WSL2 (Ubuntu)
Emulator	WinUAE (A1200, 68020 cycle-exact, AGA, AROS ROM)
Tools	Python 3 + numpy (graphics, palettes, sound, ADF, package, documentation), ffmpeg
Tests	Clang from the `ziglang` package: code check, simulator, screenshots and trailer, newspaper tests
Graphics	pixel art generated by code; illustrations prepared with an AI image generator and converted to the game's palettes
Repository	git, GitHub
AI assistants	Claude (Anthropic), GPT-6 Astra

12. Status and known limitations

- The game runs in WinUAE at 50 FPS; over a whole heat single frames are dropped (a few when the tape flies up and when the flag changes).
- **It has not been tested on a real A1200 yet**, nor with an original Kickstart 3.1.
- It uses about 1.05–1.25 MB of Chip RAM, so it needs 2 MB Chip.
- Only two engines can be heard at once (Paula has 4 channels): the two highest-placed riders still racing.
- The engine sound comes from an AI-generated recording (500 cc, 10,000 rpm), looped with `tools/import_engine.py`.
- The 4-player parallel port adapter has not been tested.
- PAL only.

To do: a practice mode, more tracks, a test on a real A1200.

13. How to update this documentation

1. Edit `docs/DOCUMENTATION.md` (English) and `docs/DOKUMENTACJA.md` (Polish) – plain Markdown: headings `#`, lists `-`, tables `| |`, images `![caption](../art/en/file.png)`, code in triple-backtick blocks.
2. If the screens changed, refresh the screenshots: `sh tools/shots/run.sh` and `LANG_EN=1 sh tools/shots/run.sh`.
3. Make the PDFs: `python tools/mkdocs.py` → `docs/SpeedwayParty-documentation.pdf` and `docs/SpeedwayParty-dokumentacja.pdf` (`--en` or `--pl` for just one). The version (git commit) and date are filled in automatically.

A `#` heading starts a new part on a new page, `##` is a chapter in the table of contents.

14. Credits and licence

Game: **b4rtiga**, with the help of artificial intelligence: **Claude** (Anthropic) and **GPT-6 Astra**.

MIT licence © 2026 b4rtiga. Source code: github.com/b4rtiga/Speedway-Party